



THE YENEPOYA INSTITUTE OF ARTS, SCIENCE,
COMMERCE AND MANAGEMENT

PROJECT-LOW LEVEL DESIGN

On

Honeypot Network for Healthcare Threat Intel

Student Name:

Mohammed Rasmil p.v

23BCCE116

Industry Mentor::

Mr.Shashank Korram

Guided by:

Ms.Munisah

Table of Content

1. Introduction	3
1.1. Scope of the Document	3
1.2. Intended Audience	3
1.3. System Overview	3
2. System Design	4
2.1. Application Design	4
2.2. Process Flow	4
2.3. Information Flow	4
2.4. Components Design	5
2.5. Key Design Considerations	5
2.6. API Catalogue	5
3. Data Design	6
3.1. Data Model	6
3.2. Data Access Mechanism	6
3.3. Data Retention Policies	6
3.4. Data Migration	7
4. Interfaces	8
5. State and Session Management	9
6. Caching	10
7. Non-Functional Requirements	11
7.1. Security Aspects	11
7.2. Performance Aspects	11
8. References	12

1. Introduction

1.1 Scope of the Document

This document provides the High-Level Design (HLD) of the Honeypot Network for Healthcare Threat Intel. It defines the system architecture, components, data flow, APIs, and non-functional requirements. The goal is to present a clear blueprint of how the system is structured and operates.

1.2 Intended Audience

- **Development Team:** To understand component interaction and integration points.
- **Project Managers:** To track architectural milestones and dependencies. **Quality Assurance (QA):** To design test cases based on process flows. **Security Analysts:** To validate the threat detection and logging mechanisms. **Industry Mentors/Guides:** For architectural review and approval.

1.3 System Overview

This system is a Python-based honeypot security solution designed to simulate a healthcare network environment. It captures unauthorized access attempts, logs threat data, and provides real-time monitoring via a web dashboard.

Key functionalities:

- Detect malicious network connections
- Log attacker details (IP, timestamp, request type)
- Provide real-time monitoring dashboard
- Simulate a fake healthcare service for deception

2. System Design

2.1. Application Design

The system follows a two-service modular architecture:

1. **Healthcare Honeypot Trap Service (TCP Listener):** A multi-threaded socket server that simulates a healthcare database listener on port 8888, captures connection metadata, logs intelligence, returns a deceptive EHR-style error response, and closes the connection.
2. **Threat Intel Dashboard Service (Flask Web Server):** A web application that reads the JSON threat log file and renders a live-updating dashboard for healthcare security operations center (SOC) analysts.

2.2. Process Flow

Flow 1 :Attacker Interaction (Threat Capture)

3. Attacker initiates a TCP connection to healthcare honeypot target-ip:8888.
4. Honeypot listener accepts the connection and extracts source IP.
5. System timestamps the event and records connection metadata.
6. System atomically appends a JSON threat record to threat_intel.json.
7. System sends deceptive response: "ACCESS DENIED: EHR Database - Unauthorized connection attempt logged. Incident ID: [GENERATED]".
8. System closes the connection.

Flow 2: Security Analyst Monitoring (Threat Intelligence)

9. Analyst navigates to http://localhost:5001 in a browser.
10. Flask server reads threat_intel.json.
11. Server renders the dashboard HTML with categorized threat data.
12. Frontend JavaScript polls /api/threats endpoint every 5 seconds.
13. Dashboard updates dynamically with new healthcare-targeted threats.

2.3. Information Flow

The diagram below illustrates the information flow between the attacker, honeypot trap engine, threat intel logger, threat intel dashboard, and the persistent JSON store.

[Information Flow Diagram: Attacker → TCP SYN to Port 8888 → Healthcare Honeypot Trap → Extract IP, Timestamp → Threat Intel Logger → Append JSON Record → threat_intel.json. Security Analyst → HTTP GET / → Threat Intel Dashboard :5001 → Read Threat Data → threat_intel.json. Healthcare Honeypot Trap → Send EHR Deception Message → Attacker. Threat Intel Dashboard → Render Dashboard / JSON API → Security Analyst.]

2.4. Components Design

Component	Technology Stack	Responsibility	Key Interfaces
Healthcare Honeypot Trap Engine	Python (socket, threading)	Listen on port 8888, simulate EHR database, handle concurrent connections, log raw threat data	start_honeypot(), handle_connection()
Threat Intel Log Manager	Python (json, datetime, threading.Lock)	Write structured threat records, ensure thread-safe file operations	log_threat(ip, status, metadata)
Threat Dashboard Backend	Flask, json	Serve web UI, provide REST API for threat intelligence	/, /api/threats (GET)
Dashboard Frontend	HTML5, Tailwind CSS, Vanilla JavaScript	Display threat table, auto-refresh, filter by IP/timestamp, export intelligence	N/A (consumes /api/threats)

2.5. Key Design Considerations

- **Healthcare Realism:** Deceptive responses mimic real EHR systems (e.g., "Epic Systems DB: Access violation").
- **Low Interaction:** Honeypot does not emulate full services, reducing attack surface and risk.
- **Concurrency Safety:** Thread-safe log writing using mutex locks to prevent corruption under high attack volume.
- **Observability:** All threat intel stored as plain-text JSON for SIEM ingestion or manual analysis.
- **Resource Efficiency:** Minimal CPU/memory footprint for deployment on edge devices or low-resource healthcare IT environments.

2.6. API Catalogue

Endpoint	Method	Description	Request Format	Response Format	Example
/	GET	Serve main threat intelligence dashboard	None	HTML (text/html)	GET http://localhost:5001/
/api/threats	GET	Return all captured threat records as JSON	None (optional ? limit=N)	JSON array of threat objects	[{"ip": "192.168.1.5", "timestamp": "2025-01-15T10:30:00", "status": "connected"}]
/api/threats/export	GET	Download threat intel as CSV	None	CSV file	GET http://localhost:5001/api/threats/export
/health	GET	Health check endpoint	None	JSON {"status": "ok", "honeypot": "running"}	GET http://localhost:5001/health

3. Data Design

3.1. Data Model

The system uses a single, flat-file data store: logs/threat_intel.json. Each line (or file array) contains a JSON object representing one healthcare-targeted attack event.

Threat Event Schema:

Field	Type	Description	Constraints	Example
timestamp	String (ISO 8601)	Time of connection attempt (UTC)	Required	"2025-01-15T14:30:22.123Z "
source_ip	String	IPv4 or IPv6 address of attacker	Required	"203.0.113.45"
status	String	Connection result	Required, enum: connected, error, timeout	"connected"
incident_id	String	Unique identifier for each threat event	Auto-generated, required	"HC-THREAT-20250115-001"

{Example JSON Threat Record:

```

{
  "timestamp": "2026-04-13T16:10:38Z",
  "source_ip": "192.168.1.37",
  "status": "error",
  "incident_id": "HC-THREAT-20260413-001" }

```

3.2. Data Access Mechanism

- The Flask backend reads the JSON file directly on each request.
- File reads are not cached by default; each API call opens and parses the file.
- Thread-safe write operations are ensured using Python's threading.Lock in the honeypot service.
- No database or ORM is used; all persistence is flat-file JSON.

3.3. Data Retention Policies

- Threat logs are stored in append-only mode; no records are deleted via the network interface.
- Log rotation is not implemented in the base system; manual archival is recommended for long-running deployments.
- Data is retained for as long as the threat_intel.json file exists on disk.

3.4. Data Migration

- No database migration is required as the system uses flat-file JSON storage.
- For SIEM integration, threat_intel.json can be ingested directly or exported via the /api/threats/export CSV endpoint.
- Future migration to a relational database would require a one-time ETL to convert JSON records to normalized schema.

4. Interfaces

Interface Type	Source Component	Target Component	Protocol	Data Format	Purpose
Network (Inbound)	Attacker Tool	Healthcare Honeypot Trap Engine	TCP	Raw bytes	Accept deception connection
Network (Outbound)	Honeypot Trap Engine	Attacker Tool	TCP	Plain text	Send EHR access denied message
File I/O	Threat Intel Log Manager	Filesystem (JSON)	File system	JSON	Persist threat intelligence records
REST API	Dashboard Frontend	Dashboard Backend	HTTP	JSON	Fetch real-time threat data
HTTP UI	Analyst Browser	Dashboard Backend	HTTP	HTML	Render threat intelligence dashboard

5. State and Session Management

Honeypot Service (Stateless)

- Each TCP connection is handled independently.
- No session data or history is stored.
- Prevents attackers from identifying patterns or system behavior.
- Simplifies design and improves scalability.

Dashboard Service (Stateless)

- No user sessions, cookies, or login states are maintained.
- Each request reads data from a JSON log file.
- Ensures up-to-date information and reduces security risks.

Attacker State (No Tracking)

- No tracking of attackers or repeat offenders.
- No blocking or adaptive responses.
- Maintains realism and avoids revealing the honeypot.

Overall Benefits

- Reduces complexity and attack surface.
- Prevents pattern detection.
- Enhances scalability and security.

6. Caching

Log Read Behavior

The system uses a direct file-read approach. The backend reads `threat_intel.json` on every request, including dashboard loads and `/api/threats` polling (every ~5 seconds). This ensures real-time accuracy and simplicity, with no cache synchronization overhead. For small datasets (<10,000 logs), disk I/O and JSON parsing remain efficient. However, performance may degrade as data volume increases due to repeated reads and parsing.

Static Asset Caching

Flask serves static assets (CSS, JavaScript) with HTTP caching headers such as `Cache-Control` and `ETag`. This allows browsers to cache frontend resources, reducing bandwidth usage and improving load times. Only dynamic API data is fetched frequently.

API Polling Model

The frontend uses periodic polling (every ~5 seconds) to fetch data from `/api/threats` in JSON format. This provides near real-time updates with a simple implementation. However, it introduces redundant requests and repeated file access, even when no new data is available.

Limitations

- No caching → repeated disk I/O
- Performance degrades with large log files
- Redundant reads during frequent polling
- Entire dataset returned per request (no pagination)

Improvement: In-Memory Caching

Introduce a lightweight in-memory cache in the backend using a Python dictionary with a TTL of ~2 seconds.

Behavior:

- If cache is valid → return cached data
- If expired → read file, update cache

Benefits:

- Reduces disk I/O significantly
- Improves response time
- Maintains near real-time accuracy

7. Non-Functional Requirements

7.1. Security Aspects

- **Honeypot Isolation:** The service runs on a dedicated port (8888) with no access to real production data, healthcare records, or system resources.
- **No Payload Execution:** The socket listener only reads the first 1024 bytes (optional) and never executes any received data.
- **Deception Integrity:** The deceptive message does not reveal the true nature of the system (no "honeypot" text, only realistic healthcare error messages).
- **Log Integrity:** JSON threat logs are append-only; no modification or deletion capability is exposed via the network.
- **Dashboard Exposure:** The dashboard (port 5001) binds to localhost only by default. In production healthcare environments, it must be behind a reverse proxy with authentication.

7.2. Performance Aspects

- **Concurrent Connections:** Support at least 50 simultaneous TCP connections using Python threads (simulating DDoS or scanning).
- **Log Write Latency:** < 10 ms per threat log entry (excluding disk I/O).
- **Dashboard Response Time:** < 200 ms for /api/threats with up to 10,000 log entries.
- **Resource Limits:** CPU usage < 5% idle, Memory < 50 MB under normal attack load.
- **Uptime:** Designed to run continuously for 30+ days without restart in healthcare SOC environments.

8. References

14. Python Software Foundation. (2024). socket — Low-level networking interface. Python 3.x Documentation.
15. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.
16. Spitzner, L. (2003). Honeypots: Tracking Hackers. Addison-Wesley Professional.
17. The HoneyNet Project. (2020). Know Your Enemy: Honeypots. Retrieved from <https://www.honeynet.org>
18. Healthcare Sector Cybersecurity Coordination Center (HC3). (2023). Threat Briefing: Ransomware Targeting Healthcare. U.S. Department of Health and Human Services.
19. Tailwind CSS. (2024). Tailwind CSS Documentation. Retrieved from <https://tailwindcss.com>